

JARVIS

VERSION 2

NEXT-GENERATION AUTONOMOUS AI OPERATING SYSTEM

Complete Development Plan & Architecture Roadmap

From personal assistant to Iron Man-level autonomous agent

CURRENT VERSION	MK37
TARGET VERSION	V2 - AUTONOMOUS
ARCHITECTURE	Multi-Provider Free AI
CORE MODEL	Gemini 2.5 Flash + Groq + Ollama
COST	100% Free
CLASSIFICATION	Personal - Prasanth Mukkiri

Generated: 07 May 2026 · Prasanth Mukkiri Industries · CLASSIFIED
J.A.R.V.I.S - JUST A RATHER VERY INTELLIGENT SYSTEM

TABLE OF CONTENTS

01	Current State - Jarvis MK37	What exists today, feature inventory, architecture
02	Completed Improvements - P1 to P3	Stability, security, and stability fixes already applied
03	Remaining Work - P4 to P7	Architecture, performance, UX polish, and testing
04	The Vision - Iron Man Jarvis	What V2 looks like and why it matters
05	5 Core Upgrades Required	ReAct loop, computer vision, parallel agents, and more
06	Stage-by-Stage Roadmap	7 stages from current to full Iron Man capability
07	Free Multi-Provider Strategy	Gemini + Groq + OpenRouter + Ollama - zero cost
08	Quota & Cost Analysis	How to run Jarvis all day for free
09	New Files to Build	Exact files, classes, and functions for V2
10	Beyond V2 - The Endgame	Levels 1-5, the long-term vision

jarvis mkg25 - what we gets today

01 - CURRENT STATE

CORE FEATURES INVENTORY

Voice Interface	Gemini 2.5 Flash Native Audio - real-time mic + speaker streaming
Long-Term Memory	6-layer system: facts, episodes, semantic vectors, KG, files, mood
Knowledge Graph	Auto-extracts entities/relations from every conversation turn
File RAG Brain	Indexes Desktop/Documents/OneDrive - semantic search over your files
Semantic Search	Gemini text-embedding-004 - 768-dim vectors, cosine similarity
Mood Tracking	Text keywords + FER webcam - 7-day emotional pattern analysis
Proactive Briefing	Startup greeting referencing memory, weather, mood, follow-ups
Follow-up Reminders	Detects intentions, persists, reminds in future sessions
PC Guardian	CPU/RAM/disk/battery/temp monitoring with alert thresholds
19 Action Modules	web search, browser, files, apps, WhatsApp, YouTube, weather, code...
Chain Runner	NLP-parsed multi-step chains with persistence across restarts
Error Recovery	Circuit breaker with exponential backoff 60->1800s
Web Dashboard	Flask real-time monitoring, news, KG visualization, memory browser
Emotion Detection	FER facial recognition from webcam - adapts Jarvis tone
Dev Agent	Builds multi-file projects from natural language description
Neural KG Visualization	Force-directed physics + animated bezier signal particles
Thread Pool	ThreadPoolExecutor(8) - bounded background workers
Atomic File Writes	os.replace() pattern - crash-safe JSON + numpy writes

OVERALL RATING BEFORE V2 WORK

Feature Completeness: 9/10 | Architecture: 6.5/10 | Security: 3.5/10 | Robustness: 5.5/10

Personal AI OS rating: 8.1/10 | Production rating: 4.5/10 | Target after V2: 9.5+/10

02 - COMPLETED IMPROVEMENTS

P1 - STABILITY

DONE	Atomic JSON writes	memory_manager, followups, knowledge_graph - os.replace() pattern
DONE	Atomic vectors.npy writes	semantic_store + file_store - write to .tmp then os.replace()
DONE	Lock _last_user_text	threading.Lock() - eliminates race condition in _build_config()
DONE	Reconnect backoff	3s->6s->12s->24s->48s->60s cap - no API hammering on outage
DONE	Face image resize cache	Keyed on pixel size (int) not float - 5x fewer PIL.resize calls
DONE	Audio stream timeout	asyncio.wait_for(__anext__(), timeout=60) - no infinite hang
DONE	Tool call timeout	asyncio.wait_for(_execute_tool, timeout=90) - stuck tools fail gracefully
DONE	typing_queue cap	deque(maxlen=50) - log display never lags by minutes
DONE	API key singleton	_api_key_cache - zero file reads after first load
DONE	Thread pool	ThreadPoolExecutor(8) - replaces 7 per-turn daemon threads
DONE	UI log line cap	_MAX_LOG_LINES=800 - Tkinter text widget never grows unbounded

P2 - SECURITY

DONE	Code executor sandbox	6 regex blocklist patterns - blocks rmtree, format, credential access
DONE	Subprocess env isolation	_SAFE_SUBPROCESS_ENV - generated code can't read your API keys
DONE	Dashboard localhost-only	before_request hook - all /api/ routes reject non-localhost requests
DONE	File path blocklist	14 sensitive segments - .ssh, .gnupg, Chrome passwords, AppData blocked
DONE	SKIP_DIRS expansion	8 new dirs - .ssh, .gnupg, Firefox, Chrome, Edge never auto-indexed
DONE	Tool param length cap	2000 char max per param - prevents prompt overflow attacks
DONE	Prompt injection guard	_sanitize_for_prompt() - strips injection phrases from memory prompts

P3 - RELIABILITY

DONE	Reconnect alert at 5+10	ERR: messages after 5 and 10 failed reconnects - user informed
DONE	KG case-insensitive merge	_merge_nodes() in _load() - Prashanth+prashanth merge correctly
DONE	KG skip write if unchanged	changed flag - disk write only when graph actually modified

03 - REMAINING WORK

P4 Architecture · P5 Performance · P6 UX · P7 Testing

P4 - ARCHITECTURE - HIGH EFFORT, HIGHEST LONG-TERM VALUE

#15	High	Split main.py	audio_pipeline.py + turn_manager.py + tool_dispatcher.py
#16	Medium	Replace print() with logging	Structured log levels across all modules
#17	Low	Named constants config.py	380, 2200, 800, 768 - all magic numbers in one file
#18	Medium	_build_config() mtime cache	Cache 6 memory reads, invalidate on file change only

P5 - PERFORMANCE - NOTICEABLE CPU/DISK SAVINGS

#19	Low	Canvas dirty-flag	Skip 60fps canvas redraw when UI state unchanged
#20	Low	TTL on semantic entries	Auto-expire embeddings older than 90 days
#21	High	Approx nearest neighbor	Bucket-based search - needed at 5000+ embedding entries

P6 - UX POLISH - MAKES JARVIS FEEL PROFESSIONAL

#22	Low	Error display in UI	Tool failures shown in Tkinter log, not just console
#23	Low	File index progress	SYS: Indexing... (247 files done) during startup scan
#24	Low	Followup snooze	snooze_until field - defer reminders by N days
#25	Medium	KG fuzzy name match	Add pal/buddy/mate + Levenshtein distance ?1 fallback

P7 - TESTING - REGRESSION SAFETY NET

#26	Medium	Unit tests memory modules	pytest - memory_manager, KG, followups, semantic_store
#27	Medium	Integration smoke test	Mock Gemini session + 5 tool call end-to-end verification

04 - THE VISION Open Minded Jarvis - Most advanced personal AI ever built

CURRENT VS TARGET CAPABILITY

CAPABILITY	MK37 NOW	V2 TARGET
Single tool call	YES	YES
Multi-step chain (linear)	YES	YES
Adaptive ReAct loop	NO	YES
Build a full project	NO	YES
Loop until task done	NO	YES
See screen + act on it	NO	YES
Fix its own errors	NO	YES
Run while you sleep	NO	YES
Parallel agent workers	NO	YES
Manage your business	NO	YES
Long tasks (hours/days)	NO	YES
Calendar + email aware	NO	YES
Multi-device sync	NO	YES
Local offline fallback	NO	YES
Plugin system	NO	YES

WHY THIS WILL BE DIFFERENT FROM AUTOGPT / BABYAGI

- Persistent identity - Jarvis knows you across months, not just one session
- Real memory - 6-layer system with KG, semantic search, mood, file RAG
- Voice-first - actual real-time audio, not text commands in a terminal
- Emotional intelligence - mood-adaptive, relationship-aware, proactive
- True computer control - sees and operates your screen like a human
- Self-healing - circuit breakers, error recovery, reconnect logic already built
- Free forever - multi-provider routing means zero cost for personal use
- Everything already built - memory, tools, dashboard carry over completely

The foundation of Jarvis MK37 is already stronger than any open-source agent project. V2 adds the autonomous loop, computer vision, and parallel workers on top - not a rewrite, an evolution.

What needs to be built to reach from Man capability

05 5 CORE UPGRADES

1. TRUE AUTONOMOUS ReAct LOOP

core/agent_brain.py

Replace chain_runner's linear execution with a Reason->Act->Observe loop that continues until the goal is genuinely achieved. Self-evaluates after each step. Detects when stuck and escalates to user only when truly needed.

- Goal-driven: keeps going until done, not just N steps
- Self-corrects: reads errors, tries alternative approaches
- Context compression every 20 steps - runs forever without token overflow
- Stuck detection: after 3 failed attempts, asks user for guidance
- Runs on background regular API - separate from voice session

2. COMPUTER VISION + CONTROL LOOP

core/computer_vision_agent.py

Close the see->act->verify loop. Current screen_process takes one screenshot. V2 continuously sees the screen, understands what's there, decides what to click or type, executes, then checks the result. Can operate any software on your PC.

- Continuous: screenshot->analyze->act->verify, repeat until done
- Uses existing computer_control + screen_process actions
- Can use Excel, Photoshop, browsers, IDEs without special integrations
- Vision model identifies UI elements, buttons, text fields precisely
- Self-verifies: confirms action had expected effect before next step

3. PARALLEL SPECIALIST AGENTS

core/agent_coordinator.py

One Jarvis doing everything sequentially is slow. V2 runs a Coordinator that spins up specialist workers for different task types, all running simultaneously.

- Coordinator: receives goal, decomposes, assigns to specialists
- Research Agent: web search, document reading, information gathering
- Code Agent: project building, debugging, testing, deployment
- Communication Agent: messages, emails, reports, notifications
- Monitor Agent: watches task progress, handles failures

4. LONG-HORIZON TASK MEMORY

memory/task_ledger.py

Current Jarvis forgets tasks between sessions. V2 maintains a persistent task ledger - active goals with full state, steps completed, steps remaining, blockers. Jarvis picks up exactly where it left off on every boot.

- Task ledger stored in memory/tasks.json - survives restarts
- Each task: goal, status, steps_done, steps_remaining, blockers, started_at
- Morning briefing includes in-progress task status automatically
- Can resume multi-day builds without any user re-explanation
- Proactive: continues tasks in background when you're not talking

5. SELF-HEALING + JUDGMENT ENGINE

agent/judgment.py

When a step fails, V2 tries alternatives before asking for help. Learns which approaches work on your specific machine and environment over time.

- Failure taxonomy: timeout / auth / not found / permission / logic error
- Per-type recovery strategies: retry / alternative tool / ask user
- Learns per-machine: what works on YOUR PC, stored in preferences
- Escalation ladder: auto-retry -> alternative -> degrade -> ask user
- Audit trail: every decision logged with reasoning for review

JARVIS V2 - CLASSIFIED DEVELOPMENT PLAN

7 stages from current Jarvis to full blown Agent capability

06 - STAGE BY STAGE ROADMAP

STAGE 1 Autonomous ReAct Engine

2-3 weeks

Replace chain_runner with goal-driven ReAct loop. Context compression. Stuck detection. This single change transforms Jarvis from tool-caller to actual agent. Every other stage becomes more powerful once this exists.

- core/agent_brain.py - ReAct loop class
- core/context_compressor.py - 20-step compression
- memory/task_ledger.py - persistent task state
- main.py - wire up AgentBrain alongside voice session

STAGE 2 Computer Vision Control Loop

2-3 weeks

Close the see->act->verify loop using existing screen_process + computer_control. Jarvis can now operate any software on your PC visually. Foundation for all software automation.

- core/computer_vision_agent.py - vision loop class
- actions/screen_processor.py - enhanced with element detection
- actions/computer_control.py - precise coordinate targeting

STAGE 3 Advanced Dev Agent

2-3 weeks

Full software lifecycle: plan -> setup environment -> write code -> run -> test -> fix -> iterate. Uses Stage 1 (ReAct) + Stage 2 (computer control) to actually build working projects.

- agent/dev_loop.py - full build lifecycle manager
- actions/code_helper.py - enhanced with test + fix loop
- actions/dev_agent.py - upgraded with real execution

STAGE 4 Parallel Agent Workers

2-3 weeks

Coordinator + specialist architecture. Research, code, communication, monitoring all run simultaneously. 10x throughput on complex tasks.

- core/agent_coordinator.py - orchestrator
- core/research_agent.py - web + document intelligence
- core/comms_agent.py - messages, emails, reports

STAGE 5 Long-Horizon Task Manager

1-2 weeks

Task ledger survives restarts. Morning briefing includes task progress. Jarvis continues multi-day builds without re-explanation.

- memory/task_ledger.py - persistent task state
- agent/proactive.py - enhanced with task status briefing
- gui/app.py - task management dashboard routes

STAGE 6 Business Automation Layer

3-4 weeks

Email reading + drafting. Calendar awareness. Document generation. Client relationship management. Report creation. Jarvis runs your professional life.

- actions/email_agent.py - Gmail/Outlook integration
- actions/calendar_agent.py - event awareness + scheduling
- actions/document_agent.py - report/proposal generation
- memory/contacts.py - professional relationship tracking

STAGE 7 The Full Vision

Ongoing

Multiple machines sharing one brain. Proactive business intelligence. Autonomous project management across days/weeks.

Plugin system for community tools.

- core/sync_engine.py - Google Drive brain sync
- core/plugin_loader.py - drop .py in plugins/, auto-register
- actions/multi_device.py - cross-machine coordination
- gui/remote_dashboard.py - phone control interface

Gemini + Groq + OpenRouter + Ollama - Zero cost forever

07 - FREE MULTI-PROVIDER STRATEGY

PROVIDER OVERVIEW

MODEL	PROVIDER	FREE LIMIT	BEST USE
Gemini 2.5 Flash	Google	1500/day	Voice, complex reasoning, memory embedding
Gemini Flash Lite	Google	1500/day	Simple agent steps, summaries - 10x cheaper quota
Gemini 2.0 Flash	Google	1500/day	Alternative routing when 2.5 Flash at limit
Groq + Llama 3.1 70B	Groq	14400/day	Agent ReAct loop steps - extremely fast, generous
Groq + Gemma 2 27B	Groq	14400/day	Backup to Llama 3.1 on same account
OpenRouter free	OpenRouter	~5000/day	DeepSeek R1, Llama 3, overflow buffer
Mistral 7B	Mistral	1000/day	Additional overflow option
Ollama local	Your PC	UNLIMITED	Always-available fallback - no internet needed

ROUTING LOGIC

The provider router selects the best available free provider based on task complexity and remaining daily quota. This runs transparently - Jarvis always has a model available.

Voice conversation	Gemini 2.5 Flash Live API	Best audio quality, real-time streaming
Memory embedding	Gemini text-embedding-004	768-dim vectors, already integrated
Complex reasoning step	Gemini 2.5 Flash (regular)	Best reasoning, use quota carefully
Simple agent step	Groq + Llama 3.1 70B	14400/day - almost never runs out
Research / summarization	Groq + Gemma 2 27B	Fast, strong, generous quota
Overflow / quota exceeded	OpenRouter free models	DeepSeek R1, Llama 3 - free tier
Offline / all limits hit	Ollama local (Llama 3.1 8B)	Zero cost, no internet, always works

DAILY CAPACITY ON FREE TIER

A complex autonomous task (100-step project build) uses roughly: 20 complex Gemini steps + 80 Groq steps. Total daily free capacity across all providers:

Gemini 2.5 Flash Lite	1500 req/day	~18 complex reasoning tasks
Groq Llama 3.1 70B	14400 req/day	~180 agent loop tasks
OpenRouter free	~5000 req/day	overflow buffer
Ollama local	Unlimited	final fallback, always available

Bottom line: running Jarvis all day doing real autonomous work - building apps, researching topics, managing tasks - is completely free with multi-provider routing. The only limit is your PC's ability to run Ollama locally as fallback.

Impact files, classes, and key functions

08 - NEW FILES TO BUILD FOR V2

CORE/

agent_brain.py	AgentBrain class - ReAct loop, run_goal(), compress_context(), is_stuck()
context_compressor.py	compress_history(steps, api_key) - Gemini summarizes last N steps
agent_coordinator.py	Coordinator class - decomposes goals, spawns workers, merges results
computer_vision_agent.py	VisionAgent - screenshot loop, element detection, act(), verify()
research_agent.py	ResearchAgent - web search chain, doc reading, fact extraction
comms_agent.py	CommsAgent - email draft, message compose, report generation
provider_router.py	ProviderRouter - tracks quota per provider, routes to best available
plugin_loader.py	load_plugins() - discovers *.py in plugins/, auto-registers tools

MEMORY/

task_ledger.py	TaskLedger - save_task(), update_task(), get_active(), resume_task()
contacts.py	Contact graph - richer than KG, stores professional context per person

AGENT/

judgment.py	JudgmentEngine - failure taxonomy, recovery strategies, escalation ladder
dev_loop.py	DevLoop - full build lifecycle: plan->code->run->test->fix->iterate

ACTIONS/

email_agent.py	email_action() - Gmail/Outlook read, draft, send via browser automation
calendar_agent.py	calendar_action() - event read, create, check availability
document_agent.py	document_action() - generate reports, proposals, summaries as .docx/.pdf

CONFIG/

config.py	All named constants - MAX_MEMORY_CHARS, EMBED_DIM, TOOL_PARAM_MAX_LEN...
providers.json	API keys for Groq, OpenRouter, Mistral - loaded by provider_router.py

TESTS/

test_memory.py	Unit tests - memory_manager, KG, followups, semantic_store
test_integration.py	Smoke test - mock Gemini, run 5 tools end-to-end, assert correct dispatch

09 - BEYOND V2 - THE ENDGAME

LEVEL 1 - Deeper Intelligence

- Self-improving memory - Jarvis notices behavioral patterns and updates its own rules
- Multi-turn tree planning - branching plans, not just linear steps
- Confidence scores - evaluates tool result quality, retries automatically
- Personality evolution - adapts communication style based on your feedback over months

LEVEL 2 - New Capabilities

- Calendar + Email - reads Gmail/Outlook, drafts replies, manages schedule
- Voice persona customization - tone, speed, formality stored in preferences
- Multi-device sync - memory/KG synced via Google Drive, same brain everywhere
- Local model fallback - Ollama when Gemini is down, zero degradation
- Scheduled autonomous tasks - cron-style proactive agent, no trigger needed

LEVEL 3 - Personalization Engine

- Behavioral fingerprinting - tracks active hours, tool preferences, topic patterns
- Adaptive verbosity - learns if you want short or detailed answers automatically
- Relationship graph enrichment - stores context per person (stressed/busy/etc.)
- Proactive suggestions - 'You usually review code on Sunday, shall I prepare the diff?'

LEVEL 4 - Production Grade

- Packaged .exe installer - no Python required, shareable with non-developers
- Plugin system - drop .py in plugins/, Jarvis discovers and registers automatically
- Web remote control - send commands from phone via secured dashboard
- Conversation export/import - portable brain, move between machines

LEVEL 5 - The Endgame Vision

- Knows your schedule, relationships, projects, mood patterns inside-out
- Acts before you ask based on behavioral fingerprint and calendar context
- Runs 24/7, self-healing when APIs fail, continues tasks across sleep cycles
- Learns from every conversation across months and years - gets smarter over time
- Portable - same brain on any device, synced, always available
- The gap between a great prototype and something truly irreplaceable - closed

THE NATURAL BUILD ORDER

Level 1 (Intelligence) -> Level 2 (Capabilities) -> Level 3 (Personalization) -> Level 4 (Production) -> Level 5 (Vision). Start with Level 1 because smarter reasoning amplifies every capability added after it. Each level compounds on the previous - the order matters.

THE MISSION

Build the most advanced personal AI that has ever existed - free, private, running on your machine, knowing you better than any commercial product ever will.